Better by Default

WPYEG · Edmonton WordPress Meetup

Sane defaults for every new WordPress site — one toggle at a time.

`a hands-on workshop · the better-by-default plugin`

Welcome to WPYEG. Tonight we build one small plugin that flips a menu of sensible defaults onto any WordPress site. Whether you write PHP daily or just manage sites, you'll leave knowing why each default matters and how to toggle it. This running text is the speaker script — in iA Presenter it stays in your notes, not on the slide.

A fresh WordPress install ships wide open

- **Usernames leak** — REST + author archives list every login name to anonymous visitors
- **Legacy XML-RPC wide open** — pingback and `system.multicall` amplifiers answer by default
- **Dead weight loads** — emoji scripts, version tags, and RSD links on every page
- **Spam surface invites** — comments, pingbacks, and trackbacks open by default

None of this is a bug. It's just defaults chosen for maximum compatibility, not for your site. Every item on this slide is a default you can flip. The rest of tonight is just: which doors, and the one line that closes each.

The one idea to take home

A “default” is just an opinionated filter behind a toggle.

```
if ( wpyeg_defaults_enabled( 'restrict_rest_user_discovery' ) ) {
    add_filter( 'rest_endpoints', $hide_users_endpoint );
}   // that's the whole pattern, repeated ~20 times
```

If you remember nothing else: a default is an `add_filter` behind an `if ( option )`. Once you see that shape, the entire plugin is just twenty variations of it. Everything else tonight is picking which filter and why.

Two words, gently: hooks & filters

- **Action** — “when you reach this moment, also DO this.” A doorbell you answer.
- **Filter** — “before you use this value, let me CHANGE it first.” A mail slot that edits the letter.

```
add_filter( 'xmlrpc_enabled', '__return_false' );
```

For the non-developers in the room: an action is a doorbell, a filter is a mail slot. WordPress is built to be interrupted at labeled moments so you never edit core. `__return_false` is a tiny built-in helper that just hands back false — perfect for switching a feature off.

Six categories of default

1. **Security** — shrink the attack surface
2. **Content** — close public spam & leaks
3. **Admin UX** — a calmer, faster dashboard
4. **Login** — sessions & credentials
5. **Branding** — own the login screen
6. **Performance** — trim the page weight

Here's the roadmap. We'll spend most of our time on security and content, then move quickly through UX, login, branding, and performance, and end at the plugin that bundles them all.

Section 1 — Security & Attack Surface

Every item in this section removes something an attacker can poke — usually in one line. The theme is simple: close what you don't use. You can't exploit an endpoint that isn't there.

Restrict REST API user discovery

`wpyeg_restrict_rest_user_discovery` · default **yes**

```
add_filter( 'rest_endpoints', function ( $ep ) {
    if ( ! is_user_logged_in() ) {
        unset( $ep['/wp/v2/users'] );
        unset( $ep['/wp/v2/users/(?P<id>[\d]+)'] );
    }
```

```
    return $ep;
} );
```

The `/wp/v2/users` endpoint hands out every author's login name to anyone — half of a brute-force guess, for free. Author enumeration is step one of most attack scripts. We close it for logged-out requests only, so the editor and integrations keep working.

Lock XML-RPC down by category

`wpyeg_xmlrpc_allow_pingbacks` / `_remote_publishing` / `_multicall` · default **no** each · (+ `_block_xmlrpc_endpoint`)

```
// each category off → remove its methods
add_filter( 'xmlrpc_methods', function ( $m ) {
  if ( ! allow( 'pingbacks' ) )
    unset( $m['pingback.ping'] );
  if ( ! allow( 'remote_publishing' ) )
    // drop wp.* metaWeblog.* mt.* blogger.*
  return $m;
} );

// multicall can't be filtered off (IXR re-adds it)
// → swap in a server that refuses it
add_filter( 'wp_xmlrpc_server_class', $refuse_multicall );
```

XML-RPC isn't one door — it's an old switchboard, and every method is a line. So we don't rip the whole thing off the wall; we unplug lines. Pingbacks go (a spam and DDoS relay), and the credential-authenticated blogging APIs go (the brute-force target) — both via a method filter. `system.multicall` is the stubborn one: unplug it and IXR plugs it right back in, so the fix is a replacement server that refuses the call. And the lines we never touch: third-party methods like Jetpack's

`jetpack.*` stay connected — which is exactly why you *don't block the endpoint* on a Jetpack site.

Keep Application Passwords available

`wpyeg_disable_application_passwords` · default **no** (available)

```
// available by default — prohibit only if opted in
if ( wpyeg_defaults_enabled(
       'disable_application_passwords' ) ) {
  add_filter(
    'wp_is_application_passwords_available',
    '__return_false'
  );
}
```

Here's the twist: this is the one we *don't* lock down. An Application Password is like a spare key cut for one app — hashed, tracked per application, and revocable on its own, no lock-change needed. That makes it the safer REST credential, and the only one core accepts for REST Basic Auth. So they stay on. Prohibit them only if site policy forbids non-interactive credentials — because switching them off doesn't stop people connecting things, it just pushes them to worse habits, like sharing the login password.

Require strong passwords

`wpyeg_require_strong_passwords` · default **yes**

```
// hooked on user_profile_update_errors
if ( strlen( $pw ) < 15 ) {
```

```
    $errors->add( 'short', 'Use 15+ characters.' );
}

// screen against known breaches (HIBP) —
// length + screening, not composition rules
if ( wpyeg_password_is_pwned( $pw ) ) {
    $errors->add( 'pwned', 'Seen in a breach.' );
}
```

The rules changed and most sites didn't notice: NIST 800–63B and OWASP now say **length plus breach screening beats composition rules**. Forcing upper/lower/number/symbol just herds everyone to `Password1!` — predictable, not strong. So we require 15+ characters and screen against Have I Been Pwned (wire the filter to its range API), enforced server-side on save and reset. The JS meter is UX; the server rule is the wall.

Remove fingerprints, add headers

`wpyeg_remove_version` / `wpyeg_security_headers` · default **yes / yes**

```
remove_action( 'wp_head', 'wp_generator' );

add_filter( 'wp_headers', function ( $h ) {
  $h['X-Content-Type-Options'] = 'nosniff';
  $h['X-Frame-Options']        = 'SAMEORIGIN';
  $h['Referrer-Policy'] =
        'strict-origin-when-cross-origin';
  return $h;
} );
```

Two quick wins. Version hiding is obscurity, not security — but it cuts automated scanner noise. The three headers are safe defaults most sites can adopt without

breaking anything. A full Content-Security-Policy is a bigger conversation for another night.

Lock REST to logged-in users (opt-in)

`wpyeg_disable_rest` · default **no**

```
add_filter( 'rest_authentication_errors',
  function ( $result ) {
    if ( ! empty( $result ) ) return $result;
    if ( ! is_user_logged_in() ) {
      return new WP_Error(
        'rest_not_logged_in', 'Auth required.',
        array( 'status' => 401 ) );
    }
    return $result;
} );
```

The nuclear option. Requiring auth for ALL REST calls stops anonymous scraping cold — but it also breaks the block editor and many blocks. That's why it ships off. Great teaching moment: not every default should default to on. Some are opt-in because they trade functionality for safety.

Section 2 — Content & Public Surfaces

These reduce spam surface and clean up the thin, duplicate URLs that bots and search engines love to crawl. Close the funnels and the leaks.

Disable comments, trackbacks & pingbacks

`wpyeg_disable_comments` · default **yes**

```
add_filter( 'comments_open', '__return_false', 20 );
add_filter( 'pings_open',    '__return_false', 20 );
add_filter( 'comments_array',
            '__return_empty_array', 20 );
// + remove_post_type_support() on init
// + remove_menu_page( 'edit-comments.php' )
// + drop the admin-bar comments node
```

For most business sites, comments are a spam magnet with little upside. We close them everywhere, hide existing threads, and drop the admin menu. If the client wants comments, leave this off — but still consider closing pingbacks and trackbacks, which are almost pure spam.

Redirect author & attachment pages

`wpyeg_disable_author_archives` / `wpyeg_redirect_attachment_pages` · **yes / yes**

```
add_action( 'template_redirect', function () {
  if ( is_author() ) {
    wp_safe_redirect( home_url('/'), 301 );
    exit;
  }
  if ( is_attachment() ) {
    // 301 to parent post, or home
  }
} );
```

Two thin-content leaks, same fix. Author archives expose login slugs; attachment pages are near-empty media wrappers. Both dilute SEO and add surface. `template_redirect` fires before a template loads — the perfect place to bounce a request. Same hook, two conditions.

Disable the emoji script

`wpyeg_disable_emojis` · default **yes**

```
add_action( 'init', function () {
  remove_action( 'wp_head',
    'print_emoji_detection_script', 7 );
  remove_action( 'wp_print_styles',
    'print_emoji_styles' );
  // ...admin + feed + mail variants too
  add_filter( 'emoji_svg_url', '__return_false' );
} );
```

Core injects an emoji-detection script and inline CSS on every page load, plus a DNS-prefetch hint. Modern browsers render emoji natively, so this is pure dead weight. Small win, but it’s on literally every page — a good example of a “why is this even on?” default.

Section 3 — Admin UX & Login Sessions

Now the quality-of-life defaults. These are more about the daily experience and session safety than raw hardening.

Faster search, quieter admin bar

`wpyeg_title_only_admin_search` / `wpyeg_frontend_admin_bar_behavior` · **no / ''**

```
// title-only admin search — narrow the COLUMNS
add_filter( 'post_search_columns',
  function ( $cols, $s, $q ) {
    if ( is_admin() && $q->is_main_query() )
        return array( 'post_title' );  // titles only
    return $cols;                       // front-end untouched
  }, 10, 3 );

// hide bar for non-admins
add_filter( 'show_admin_bar', fn( $s ) =>
  current_user_can('manage_options') ? $s : false );
```

Search the admin post list on a big site and WordPress reads every word of every post — like finding a book by reading the whole library. Title-only search checks just the spines, and it's far faster. The craft is in the *how*: `post_search_columns` (WP 6.2+) narrows the columns instead of rewriting the whole SQL clause, so core's term parsing and the logged-out password guard stay intact. Scope the filter; don't bulldoze the query.

Right-size the login session

`wpyeg_remember_me_days` / `wpyeg_session_regular_hours` · default **5 / 0**

```
add_filter( 'auth_cookie_expiration',
  function ( $exp, $uid, $remember ) {
    if ( $remember ) {
      return 5 * DAY_IN_SECONDS;
```

```
    }
    return 12 * HOUR_IN_SECONDS;
  }, 10, 3 );
```

Core's "Remember Me" lasts 14 days — often too long. Cap it, optionally shorten regular logins, or hide the checkbox entirely for shared machines. One filter does all three. WordPress ships handy time constants like `DAY_IN_SECONDS`, so you never hand-count seconds.

Section 4 — Branding & Performance

The last pair: a branding touch on the login screen, then two performance levers to shave the last bit of weight off every page.

Own the login screen

`wpyeg_login_logo_behavior` · default **keep_default** (keep / remove / unlink / replace)

```
// remove, unlink, or replace — a deliberate choice
add_action( 'login_head', $logo_css ); // hide or swap image

// any change points the link home (no separate toggle)
add_filter( 'login_headerurl', 'home_url' );
add_filter( 'login_headertext', fn() =>
            get_bloginfo( 'name' ) );
```

The login page is the site's front door — and by default the welcome mat links to someone else's house. That little WordPress "W" on `wp-login.php` points out to

wordpress.org: a subtle trust leak on the one page where trust matters most. Still, repainting a client's front door uninvited is rude, so the default is to **leave it alone**. Removing, unlinking, or replacing the logo is an opt-in — and whichever you choose, the link always points home. Swap in a background-image to drop in the client's own logo. Clients always notice this one.

Throttle Heartbeat, defer scripts (opt-in)

`wpyeg_throttle_heartbeat` / `wpyeg_defer_scripts` · default **no / no**

```
add_filter( 'heartbeat_settings', fn( $s ) => {
  $s['interval'] = 60; return $s;
} );

add_filter( 'script_loader_tag',
  function ( $tag, $handle ) {
    // add ' defer' (skip jquery-core)
    return $tag;
}, 10, 2 );
```

Two opt-in levers. The Heartbeat API polls `admin-ajax` every 15–60s — throttle it to ease shared hosting. Deferring non-critical scripts stops them blocking render. Both are off by default because deferring can break plugins expecting synchronous jQuery. Test before shipping — that's why they're opt-in.

Three things a plugin can't toggle

```
define( 'DISALLOW_FILE_EDIT', true );  // no in-dashboard code editor
define( 'AUTOSAVE_INTERVAL', 120 );    // gentler autosave (seconds)
define( 'WP_POST_REVISIONS', 10 );     // cap revision-table bloat
```

- **Kills the theme/plugin editor** — a stolen admin login can't rewrite your PHP
- **Writes to the DB less often** — fewer autosave revisions during long edits
- **Keeps revisions in check** — ten per post instead of unbounded growth

Some defaults live in `wp-config.php`, above the plugin layer, because they must load before plugins do. They can't be options — so document them as manual steps in your onboarding checklist and put them in your standard wp-config template.

How the plugin is built

1. **schema()** — one array: every setting, its default, type & group. The single source of truth.
2. **settings page** — loops the schema to render toggles under Settings → Better by Default.
3. **bootstrap()** — for each *enabled* key, wires its `add_filter` / `add_action` to the right hook.

```
$stored = get_option( 'wpyeg_better_by_default' );   // read once
foreach ( wpyeg_defaults_schema() as $key => $field ) { /* render + w
```

The design lesson is a data-driven plugin. Adding a new default equals one array entry plus one `if`-block in bootstrap — no new settings-page code. That's the pattern to steal for your own projects.

Hands-on: install & flip switches

1. **Upload the plugin** — Plugins → Add New → Upload Plugin → choose `better-by-default.zip` → Activate
2. **Open the settings** — Settings → Better by Default; every toggle grouped by category
3. **Verify a default** — visit `/wp-json/wp/v2/users` logged out → 401 or empty, not a list of usernames
4. **Toggle & re-check** — flip a switch off, reload, watch the behavior change

```
# prefer the terminal?
wp plugin install ./better-by-default.zip --activate
```

Do this live if there's a sandbox. The `/wp-json/wp/v2/users` check is the crowd-pleaser — the before/after is instantly visible. For the terminal crowd, the WP-CLI one-liner installs and activates from the zip in one shot; swap the local path for a URL if the zip is hosted.

Your turn: add one new default

Goal: disable the WordPress dashboard "Welcome" panel. Two small edits — no new settings-page code.

```
// 1) add a schema entry in wpyeg_defaults_schema()
'hide_welcome_panel' => array(
    'default' => 'yes', 'type' => 'toggle', 'group' => 'ux',
    'label' => 'Hide dashboard welcome panel',
),

// 2) wire it inside wpyeg_defaults_bootstrap()
```

```
if ( wpyeg_defaults_enabled( 'hide_welcome_panel' ) ) {
    remove_action( 'welcome_panel', 'wp_welcome_panel' );
}
```

A great confidence-builder: it proves the data-driven pattern. Touch two spots and a real feature toggles. If time is short, walk it through verbally instead of live.

The cheat sheet — defaults that ship ON

Default	Core hook	Category
Restrict REST user discovery	`rest_endpoints`	Security
Lock down XML-RPC by category	`xmlrpc_methods` / `wp_xmlrpc_server_class`	Security
Require strong passwords (15+ / breach-screened)	`user_profile_update_errors`	Security
Remove version + security headers	`wp_generator` / `wp_headers`	Security
Disable comments & pingbacks	`comments_open` / `pings_open`	Content
Redirect author + attachment pages	`template_redirect`	Content / SEO
Disable emoji script	`init` (remove_action)	Performance

This is your screenshot slide — everything on-by-default in one view, mapped to the core hook. Two deliberate *non*-defaults worth calling out: Application

Passwords stay **available** (the safer REST credential), and the login logo is **left alone** unless you opt in — both are choices, not oversights.

Thanks, WPYEG!

Take the plugin, fork it, teach it. A default is just an opinionated filter behind a toggle.

`Files: better-by-default.zip · wordpress-default-settings.md`

Questions? License GPL-2.0-or-later — ship it.

Hand out the zip and the reference doc. Invite everyone to add their own favorite default to the schema and share it back with the group. Thanks for having me.